SpMM–A1 / GNN sparse–dense aggregation: correctness and memory model

Weight: 10% of the course grade | Due date: to be announced

Model and scope

Study weighted neighborhood-feature aggregation Y=A×X, where A is CSR (R×C) and X is a dense row-major feature matrix (C×F). For each row r and feature f, sum values[k] × X[col_idx[k],f] over k=row_ptr[r] through row_ptr[r+1]−1. This is the common SpMM aggregation primitive of many GNN layers, NOT an entire GCN with normalization, learned linear weights or activation.

Required activities

Stage	Student evidence
Semantics — 1 mark	Define R, C, NNZ, F, row-major X/Y, element types and the weighted aggregation operation.
Golden tiny case — 3 marks	For A (3×4), rows are [(c0,2),(c3,1)], [(c1,−1),(c2,3)], [(c0,4)]; X rows are [1,2], [2,1], [3,0], [4,−1]. Compute Y manually; run --selftest.
Two layouts — 1 mark	Run local and permuted at equal dimensions; verify matching checksums; explain how both X rows and col_idx are relocated.
Footprint — 2 marks	Calculate bytes of row_ptr, col_idx, values, X, Y and expected at R=32,768, C=4R, 8 NNZ/row, F=32.
Traffic vs storage — 2 marks	Compute logical feature reads NNZ×F×8 B per invocation; explain why that is not DRAM bytes.
Hypothesis — 1 mark	Propose one neighborhood/feature reuse hypothesis and a measurable falsification experiment.

Important interpretation

Each neighbor lookup retrieves F consecutive feature values. Explain how cache-line utilization, graph ordering, reuse, and feature-table size affect locality. A scrambled layout is not guaranteed to increase every cache-miss event.

Status

The supplied code implements weighted CSR neighborhood aggregation, not an entire GNN layer. This submission is assessed on correctness, data representation and memory analysis. Group policy and deadline will be announced by the instructor.

Exact build and execution commands

```
g++ -O3 -g -fno-omit-frame-pointer -std=c++17 -Wall -Wextra \
  SpMM_GNN_Aggregation_Benchmark.cpp -o spmm_gnn
./spmm_gnn --selftest
./spmm_gnn --pattern local --rows 32768 --nnz-per-row 8 \
  --features 32 --iters 100
./spmm_gnn --pattern permuted --rows 32768 --nnz-per-row 8 \
  --features 32 --iters 100
sha256sum SpMM_GNN_Aggregation_Benchmark.cpp \
  di_common.hpp di_kernel_pmu.hpp
```

Baseline experiment dimensions

Variable	Fixed baseline
R / C / NNZ	32,768 / 131,072 / 262,144
Feature count F	32
Type	FP64 features/weights and FP64 output
X feature matrix	32 MiB
Timed work	One complete aggregation over all output rows/features

Submission (SpMM_A1_<studentID>)

- report.pdf (≤3 pages): the manual 3×2 output, CSR representation, byte ledger, logical demand expression, and one hypothesis with test.
- terminal.txt: build, selftest, local and permuted outputs, source checksums; include altered source only if modified.

Acceptance rules

Both baseline patterns must pass the same independent logical oracle and have matching checksums. A correctness PASS does not establish a speedup; raw runtimes are not automatically comparable across different F or NNZ. Numerical reductions have a fixed order in this reference kernel; future optimizations must define their accuracy standard.

Access-pattern interpretation

Explain why fetching F contiguous features for one neighbor may use cache lines more effectively than fetching isolated feature elements. State one counterexample involving an oversized working set or poor temporal reuse.